Day 6

What is Machine Learning?

Big Picture • Concepts • Intuition • First ML Model

■ AI vs ML vs DL | ■ Types of ML | ■ ML Workflow | ■ scikit-learn | ■ Lab

1 Section 1 – AI vs ML vs Deep Learning

Before we write a single line of code, let's build a crystal-clear mental picture of what Artificial Intelligence, Machine Learning, and Deep Learning actually mean — and how they relate to each other.

The 'Russian Dolls' Analogy

Think of three Russian nesting dolls. The biggest doll is **AI**. Open it and you find **ML** inside. Open that and you find **Deep Learning** at the centre. Every DL system is also an ML system. Every ML system is also an AI system — but not the other way around.

■ AI (Artificial Intelligence)	■ ML (Machine Learning)	■ Deep Learning
The biggest umbrella. Any technique that makes a machine appear smart.	A subset of AI where machines learn patterns from data — no manual rules.	A subset of ML using layered neural networks inspired by the brain.
Chess engine, expert systems, voice assistants, self-driving cars	Spam filter, fraud detection, recommendation engines	Image recognition, ChatGPT, voice-to-text, style transfer

▲ Every column is a subset of the one on its left.

1A — Artificial Intelligence (AI)

AI is the broad science of creating machines that can perform tasks normally requiring human intelligence — reasoning, planning, understanding language, recognising faces, and so on.

■ Real-Life AI Examples You Already Know

- Chess computers (Deep Blue beat world champion in 1997)
- Voice assistants — Alexa, Siri, Google Assistant
- Self-driving cars that read road signs and avoid pedestrians
- Google Translate converting one language to another
- Your phone predicting the next word when you type

Early AI (before ~2000) was mostly **rule-based**: programmers hand-coded thousands of IF-THEN rules. E.g., IF email contains 'You have won a prize' THEN mark as spam. This works for simple cases but breaks when the world is complex and messy.

1B — Machine Learning (ML)

ML flips the programming model. Instead of humans writing the rules, we give the machine **lots of examples** and let it **discover the rules itself**.

■ The Key Shift — Old vs New Way

OLD WAY (Traditional Programming): Data + Rules → Answers

NEW WAY (Machine Learning): Data + Answers → Rules (Model)

Example: Spam detection. Old way — a developer writes: *if the word 'lottery' appears, mark as spam*. New way — show the machine 1 million emails labelled spam/not-spam and let it figure out the pattern. The ML model learns that spammers also use words like 'winner', 'claim', 'free gift', and many more — patterns a human might miss.

1C — Deep Learning (DL)

Deep Learning is ML with a special kind of model called a **Neural Network**. These networks are loosely inspired by the neurons in your brain. 'Deep' means the network has **many layers** — each layer learns increasingly abstract features.

■ How a Neural Network Sees a Cat Photo

Layer 1 — detects basic edges and corners in the pixels
Layer 2 — combines edges into shapes (circles, curves)
Layer 3 — combines shapes into parts (ears, eyes, whiskers)
Layer 4 — combines parts to recognise 'cat'
You don't program any of this — the network learns it from millions of cat photos!

Deep Learning powers ChatGPT, image recognition on your phone, YouTube's content recommendations, voice-to-text, and translation. Today we focus on regular ML — deep learning comes later in the course.

2 Section 2 – Types of Machine Learning

There are three main learning styles — think of them as three different kinds of school environments.

2A — Supervised Learning ■ (Learning with a Teacher)

In supervised learning, every training example comes with the correct answer (called a **label**). The model learns to map inputs to outputs by seeing thousands of labelled examples.

■ Classroom Analogy

Imagine a student learning to identify fruits. The teacher holds up an apple and says 'This is an apple'. Then a banana — 'This is a banana'. After seeing enough examples WITH labels, the student can identify new fruits independently. This is EXACTLY how supervised learning works!

Task	Input (X)	Output / Label (y)
Email spam filter	Email text	Spam / Not Spam
Predict house price	Size, location, rooms	Price in rupees
Medical diagnosis	X-ray image	Disease / No Disease
Student result	Study hours, attendance	Pass / Fail
Sentiment analysis	Review text	Positive / Negative

▲ Every row shows a supervised learning problem. The model trains on rows with known outputs.

Supervised learning has two sub-types:

■ **Regression:** predict a continuous number (price, temperature, score)

■■ **Classification:** predict a category (spam/not-spam, pass/fail, cat/dog)

2B — Unsupervised Learning ■ (Learning without a Teacher)

Here, the data has **no labels**. The machine must find hidden patterns, groups, or structures entirely on its own. There is no 'right answer' provided.

■■ Shopping Analogy

A new store owner doesn't know her customers yet. She looks at purchase histories and notices: Group A buys baby products, Group B buys sports gear, Group C buys luxury items. She never told the algorithm these groups — it discovered them! She can now target ads to each group. This is clustering — unsupervised learning.

Common unsupervised techniques:

■ **Clustering:** group similar data points (customer segmentation, document grouping)

■ **Dimensionality Reduction:** simplify data by keeping only important features

■ **Anomaly Detection:** find unusual data points (credit card fraud, defective products)

2C — Reinforcement Learning ■ (Learning by Trial, Reward & Punishment)

An agent learns by **interacting with an environment**. It tries actions, receives rewards for good actions and penalties for bad ones, and gradually learns the best strategy.

■ Video Game Analogy

Imagine teaching a robot to play a game — but you never explain the rules. The robot presses random buttons. When it scores a point → reward (+1 point). When it loses a life → penalty (-1 point). Over thousands of games it learns the perfect strategy entirely by trial and error. This is reinforcement learning!

Real-world applications: AlphaGo (beat world champion at Go), self-driving car simulation training, robot arm learning to grasp objects, personalised recommendation systems, algorithmic trading.

Feature	Supervised	Unsupervised	Reinforcement
Labels needed?	■ Yes	■ No	■ Rewards only
Teacher present?	■ Yes	■ No	Indirect (environment)
Goal	Predict label	Find structure	Maximise reward

Difficulty (beginner)	■■ Easy	■■■ Medium	■■■■ Hard
Today's focus?	■ YES	Later in course	Later in course

▲ Quick comparison of the three ML types.

3 Section 3 – The ML Workflow (5 Steps)

Every ML project — from predicting house prices to detecting cancer — follows the same 5-step pipeline. Master this pipeline and you understand 80% of what a data scientist does.

1 Collect and Understand Data

Gather the raw data you need. Understand what each column means.

Ask: Do we have enough data? Is it relevant? Where did it come from?

Example: Collect 1,000 students' records — hours studied & pass/fail result.

2 Clean and Prepare the Data

Real data is messy! Fix missing values, remove duplicates, fix typos.

Feature engineering: create new useful columns from existing ones.

Split data into Training set (80%) and Testing set (20%).

3 Train a Model

Choose an algorithm (e.g., Logistic Regression, Decision Tree).

Feed the training data into the algorithm using fit().

The model adjusts its internal parameters to learn the pattern.

4 Evaluate — Is It Accurate?

Test the trained model on the test set (data it has NEVER seen).

Measure accuracy, precision, recall depending on the problem type.

If accuracy is low, go back and improve data or try a different algorithm.

5 Deploy — Use It in the Real World

Package the model into an app, API, or website.

Monitor performance over time — the world changes, so must the model.

Example: Deploy as a web app where students enter hours and get prediction.

■■ The 'Garbage In, Garbage Out' Rule

The single most important factor in ML success is DATA QUALITY.

A brilliant algorithm on bad data gives terrible results.

A simple algorithm on great data often beats a complex one on bad data.

Steps 1 and 2 typically consume 70-80% of a data scientist's time!

4 Section 4 – Introduction to scikit-learn (sklearn)

scikit-learn (imported as **sklearn**) is the most popular and beginner-friendly Machine Learning library for Python. It was created in 2007 and is now used by thousands of companies and researchers worldwide.

■ Why scikit-learn?

- 100+ ML algorithms all accessible with the SAME simple API
- Works seamlessly with NumPy and pandas
- Excellent documentation with examples for every function
- Free, open-source, and battle-tested in production
- Used at Spotify, Booking.com, JP Morgan, and many more

The Three Magic Methods — The Whole sklearn API in 3 Words

No matter which algorithm you use — whether it's a simple linear model or a complex random forest — sklearn always provides the same three methods:

Method	What It Does	Analogy
`.fit(X, y)`	Train the model on your data	Student studies textbook
`.predict(X)`	Make predictions on new data	Student takes the exam
`.score(X, y)`	Measure accuracy (0.0 to 1.0)	Teacher grades the exam

▲ Learn these 3 methods and you can use ANY sklearn algorithm immediately.

Installing and Importing sklearn

```
# Install (run once in terminal):
pip install scikit-learn

# Basic imports you will use every session:
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# sklearn also includes toy datasets for practice:
from sklearn.datasets import load_iris, load_digits
```

The Universal sklearn Pattern (Template)

Every sklearn project follows this exact skeleton — copy-paste it for any problem:

```
# STEP 1: Import
from sklearn.some_module import SomeAlgorithm
from sklearn.model_selection import train_test_split

# STEP 2: Prepare Data
X = feature_data          # Input columns (2D array / DataFrame)
y = labels                # Target column (1D array / Series)

# STEP 3: Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# STEP 4: Train Model
model = SomeAlgorithm()   # Create an instance of the algorithm
model.fit(X_train, y_train)  # Train on training data

# STEP 5: Predict and Evaluate
predictions = model.predict(X_test)    # Predict on test data
accuracy    = model.score(X_test, y_test)  # How accurate?
print(f'Accuracy: {accuracy:.2%}')     # e.g., Accuracy: 92.50%
```

5 Section 5 – LAB: Predict Student Pass / Fail

Time to build your **very first Machine Learning model**! We will predict whether a student passes or fails based on how many hours they studied. This is a **supervised classification** problem.

■ Lab Objectives

1. Understand the full ML workflow end-to-end (all 5 steps)
2. Create a dataset manually and load it with pandas
3. Train a Logistic Regression classifier using sklearn
4. Evaluate accuracy and interpret results
5. Make predictions for new students

Step 1 — Understand the Problem

Question: Given the number of hours a student studies, can we predict whether they will Pass (1) or Fail (0)?

Input (Feature X): Hours Studied (e.g., 1, 3, 5, 7)

Output (Label y): Pass=1, Fail=0

Our Training Data:

Student	Hours Studied	Result (Label)
Student 1	1 hour	Fail (0)

Student 2	2 hours	Fail (0)
Student 3	3 hours	Fail (0)
Student 4	4 hours	Pass (1)
Student 5	5 hours	Pass (1)
Student 6	6 hours	Pass (1)
Student 7	7 hours	Pass (1)
Student 8	8 hours	Pass (1)
Student 9	9 hours	Pass (1)
Student 10	10 hours	Pass (1)

▲ Our tiny labelled dataset. Students studying ≥ 4 hours pass in this example.

Step 2 — Full Lab Code (Complete, Runnable)

Copy this exactly into a file called `day6_lab.py` and run it:

```
# ■■ Day 6 Lab: Predict Student Pass/Fail ■■■■■■■■■■■■■■■■■■■■ Part 1
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# ■■ 1. Create the Dataset ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
data = {
    'hours_studied': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'pass_fail':     [0, 0, 0, 1, 1, 1, 1, 1, 1,  1]
}
df = pd.DataFrame(data)
print('Our Dataset:'); print(df)

# ■■ 2. Separate Features (X) and Labels (y) ■■■■■■■■■■■■■■■■■■■■■■
X = df[['hours_studied']]   # 2D — sklearn needs 2D array for X
y = df['pass_fail']         # 1D — target labels

# ■■ 3. Split into Training and Testing Sets ■■■■■■■■■■■■■■■■■■■■■■
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print(f'Train: {len(X_train)} rows | Test: {len(X_test)} rows')
```

```
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Part 2
# ■■ 4. Create and Train the Model ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
model = LogisticRegression()
model.fit(X_train, y_train)     # THE magic line — learning happens!
print('Model trained successfully!')

# ■■ 5. Evaluate on Test Set ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
y_pred   = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2%}')
print(classification_report(y_test, y_pred,
      target_names=['Fail', 'Pass']))

# ■■ 6. Predict for Brand-New Students ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
new_students = [[2], [5], [8]]
predictions  = model.predict(new_students)
labels = {0: 'FAIL', 1: 'PASS'}
for hrs, pred in zip(new_students, predictions):
    print(f'Studied {hrs[0]} hrs  →  {labels[pred]}')
```

Step 3 — Understanding Every Line

Code	What it does / Why
`import numpy as np`	NumPy handles numerical arrays. sklearn works on arrays.
`import pandas as pd`	Pandas creates DataFrames — like Excel tables in Python.
`from sklearn.linear_model import LogisticRegression`	Import the classification algorithm we will use.
`df = pd.DataFrame(data)`	Convert our Python dictionary into a proper table.
`X = df[['hours_studied']]`	X = inputs. Double brackets [[]] give us a 2D array (required by sklearn).
`y = df['pass_fail']`	y = labels. Single bracket gives 1D array (correct for labels).
`train_test_split(... test_size=0.2)`	Randomly split data: 80% to train, 20% to test. Never train on test data!
`random_state=42`	Fixes the random seed so the split is reproducible (same every run).
`model = LogisticRegression()`	Create an untrained model object. Think of it as a blank brain.
`model.fit(X_train, y_train)`	THE most important line. The model learns patterns from training data.
`model.predict(X_test)`	Apply learned patterns to new/unseen data to generate predictions.
`accuracy_score(y_test, y_pred)`	Compare predictions to real answers. Returns a number 0.0 to 1.0.
`classification_report(...)`	Detailed breakdown: precision, recall, F1 per class.
`model.predict([[2],[5],[8]])`	Predict pass/fail for students who studied 2, 5, and 8 hours.

Step 4 — Expected Output

```
Our Dataset:
   hours_studied  pass_fail
0              1          0
1              2          0
2              3          0
3              4          1
...          ...        ...

Training samples: 8, Test samples: 2
Model has been trained!
Accuracy: 100.00%

              precision    recall  f1-score   support
        Fail       1.00      1.00      1.00         1
        Pass       1.00      1.00      1.00         1
    accuracy                           1.00         2

Student who studied 2 hrs: FAIL
Student who studied 5 hrs: PASS
Student who studied 8 hrs: PASS
```

■ Why 100% Accuracy Here?

With only 10 data points and a clear pattern (>= 4 hrs = pass), the model learns perfectly. In real life, data is noisy and 100% accuracy is suspicious! A 85-95% accuracy on a real dataset would be excellent.

6 Section 6 – Challenge Exercises

Complete these exercises to solidify your Day 6 understanding:

- ■ **Easy:** Add a 2nd feature — 'attendance_percent' — and retrain.
- ■ **Easy:** Change test_size to 0.3. Does accuracy change? Why?
- ■ **Medium:** Replace LogisticRegression with DecisionTreeClassifier and compare.
- ■ **Medium:** Add 20 more rows of data with realistic noise. What happens to accuracy?
- ■ **Hard:** Add 'sleep_hours' and 'previous_score'. Measure which feature matters most.
- ■ **Hard:** Find a real dataset on Kaggle, apply the same 5-step workflow.

7 Section 7 – Key Concepts Glossary

Term	Simple Definition
Algorithm	A step-by-step method a machine uses to learn from data.
Model	The output of training — the 'brain' that makes predictions.
Feature (X)	An input variable used to make predictions (e.g., hours studied).
Label (y)	The correct answer / output variable (e.g., pass/fail).
Training Set	Data the model learns from (typically 80% of your data).
Test Set	Data the model is evaluated on — it must NEVER see this during training.
Accuracy	Fraction of correct predictions. 0.90 = 90% correct.
fit()	Train the model on data. The learning happens here.
predict()	Use the trained model to make predictions on new data.
score()	Returns accuracy — how well the model performs on given data.
Overfitting	Model memorises training data but fails on new data (too specific).
Underfitting	Model is too simple to capture the pattern (too general).
Classification	Predicting a category/class (spam/not-spam, pass/fail).
Regression	Predicting a continuous number (house price, temperature).
Logistic Regression	Despite the name, it is a CLASSIFICATION algorithm — not regression!

8 Section 8 – Day 6 Summary & What's Next

Today You Mastered:

- **AI vs ML vs DL:** nested concepts — DL ⊂ ML ⊂ AI
- **Supervised Learning:** labelled data → learn to predict
- **Unsupervised Learning:** no labels → find hidden patterns
- **Reinforcement Learning:** reward/punishment → optimal strategy
- **ML Workflow:** 5 steps — Collect → Clean → Train → Evaluate → Deploy
- **scikit-learn:** fit(), predict(), score() — the universal API
- **First ML Model:** Logistic Regression to predict student pass/fail

■ Coming Up — Day 7 Preview

Day 7: Linear Regression — Predicting Continuous Numbers

- Predict house prices, student scores, sales revenue
- Understand the maths behind the straight line (y = mx + c)
- Visualise regression with matplotlib
- Lab: Predict salary from years of experience

Remember the Golden Rule of ML:

"More good data always beats a more complex algorithm. Understand your data first, model second." — Andrew Ng

■■

Day 6 Material — Beginner ML Course • 6 Hours • No Maths — Only Intuition!